



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA

MÁSTER UNIVERSITARIO EN INGENIERÍA
DE ANÁLISIS DE DATOS, MEJORA DE
PROCESOS Y TOMA DE DECISIONES

TRABAJO

Programación de la producción

Rafael Macian Castilla
Patricia Rodrigo Barrio

José Ramón Mena Pérez
Víctor Rodríguez Albendea

Índice

1. Introducción	3
2. Descripción del problema	3
3. Modelo matemático base	4
3.1. Parámetros y variables	4
3.2. Función objetivo	4
3.3. Restricciones	4
3.4. Resolución del modelo base	4
4. Metodología	5
4.1. Enfoques de resolución	5
4.2. Estructura de extensiones	5
4.3. Generación de datos	6
4.4. Métrica de comparación	7
5. Algoritmo Genético	7
6. Modelo exacto: CP-SAT	9
7. Iterated Greedy	10
8. Extensión 0: Modelo base $R \parallel C_{\text{máx}}$	11
8.1. Resultados computacionales	11
9. Extensión 1: Release dates r_j	12
9.1. Nuevos parámetros y restricciones	12
9.2. Adaptación de los métodos	13
9.3. Resultados computacionales	13
10. Extensión 2: Tiempos de setup dependientes de la secuencia	14
10.1. Nuevos parámetros y restricciones	14
10.2. Adaptación de los métodos	14
10.3. Resultados computacionales	14
11. Extensión 3: Relaciones de precedencia	15
11.1. Nuevos parámetros y restricciones	15
11.2. Adaptación de los métodos	15
11.3. Resultados computacionales	16
12. Extensión 4: Disponibilidad de recursos de personal	17
12.1. Nuevos parámetros y restricciones	17
12.2. Adaptación de los métodos	17
12.3. Resultados computacionales	17
13. Extensión 5: Tardanza ponderada $\sum w_j T_j$	18
13.1. Nuevos parámetros y restricciones	18
13.2. Adaptación de los métodos	18
13.3. Resultados computacionales	19
14. Conclusiones	20

1. Introducción

La planificación y programación de la producción constituye una parte fundamental dentro de la Dirección de Operaciones, ya que permite organizar el uso de los recursos productivos con el objetivo de fabricar los productos requeridos de forma eficiente y cumpliendo los plazos establecidos. En particular, la programación de la producción se centra en la asignación y secuenciación de trabajos en los recursos disponibles, determinando dónde y cuándo debe ejecutarse cada trabajo.

Uno de los problemas más relevantes en este ámbito es el denominado problema de *scheduling*, cuyo objetivo es optimizar el uso de los recursos minimizando distintos criterios de rendimiento, como el tiempo total de finalización, los retrasos o el tiempo de flujo. Estos problemas se caracterizan por la existencia de un conjunto de trabajos que deben procesarse en un conjunto de máquinas, siendo necesario decidir tanto la asignación de los trabajos a las máquinas como el orden en que se ejecutan.

Entre los distintos entornos productivos, uno de los más generales es el de máquinas paralelas no relacionadas, en el cual cada trabajo puede procesarse en cualquiera de las máquinas disponibles, pero el tiempo de proceso depende de la máquina seleccionada. Este entorno resulta útil para representar situaciones reales en las que las máquinas no son equivalentes y un mismo trabajo puede requerir tiempos distintos según dónde se procese (Fanjul-Peyro & Ruiz, 2010; Vallada & Ruiz, 2011).

En este trabajo se estudia un problema de programación de la producción en máquinas paralelas no relacionadas, partiendo de un modelo base cuyo objetivo es minimizar el *makespan*. A partir de este modelo, se introducen distintas extensiones que permiten analizar situaciones más complejas, incorporando de forma progresiva restricciones como fechas de disponibilidad, tiempos de preparación dependientes de la secuencia, relaciones de precedencia, disponibilidad de recursos de personal y, finalmente, un objetivo basado en la tardanza ponderada (Afzalirad & Rezaeian, 2016; Allahverdi, 2015; Allahverdi et al., 2008).

2. Descripción del problema

El modelo considerado parte de una matriz de tiempos de proceso, donde cada elemento p_{ij} representa el tiempo necesario para procesar el trabajo j en la máquina i . Esta matriz define todas las posibles asignaciones entre trabajos y máquinas.

El conjunto de trabajos se denota como $j = 1, 2, \dots, N$ y el conjunto de máquinas como $i = 1, 2, \dots, M$. En el caso objeto de estudio, se dispone de $N = 50$ trabajos y $M = 5$ máquinas paralelas no relacionadas. Cada trabajo debe procesarse en una única máquina, y cada máquina puede ejecutar varios trabajos de forma secuencial.

El objetivo es determinar la asignación de los trabajos a las máquinas de forma que se minimice el *makespan*, es decir, el instante de finalización del último trabajo. Por tanto, el modelo base considerado corresponde a:

$$R \parallel C_{\max}$$

3. Modelo matemático base

3.1. Parámetros y variables

Como parámetros del modelo se tiene:

p_{ij} : tiempo de proceso del trabajo j en la máquina i , $i = 1, \dots, M$, $j = 1, \dots, N$

En cambio, como variables de decisión se define:

$$x_{ij} = \begin{cases} 1 & \text{si el trabajo } j \text{ se asigna a la máquina } i \\ 0 & \text{en caso contrario} \end{cases} \quad C_{\text{máx}} \geq 0 : \text{makespan del sistema}$$

3.2. Función objetivo

$$\text{mín } C_{\text{máx}}$$

3.3. Restricciones

Cada trabajo debe ser asignado exactamente a una máquina:

$$\sum_{i=1}^M x_{ij} = 1 \quad \forall j = 1, \dots, N$$

El makespan debe ser mayor o igual que el tiempo total de procesamiento de los trabajos asignados a cada máquina:

$$\sum_{j=1}^N p_{ij} x_{ij} \leq C_{\text{máx}} \quad \forall i = 1, \dots, M$$

Naturaleza binaria de las variables de asignación:

$$x_{ij} \in \{0, 1\} \quad \forall i = 1, \dots, M, \quad j = 1, \dots, N$$

3.4. Resolución del modelo base

El modelo matemático descrito ha sido implementado y resuelto utilizando el complemento *OpenSolver* en Microsoft Excel. Los datos de entrada corresponden a la matriz de tiempos de proceso p_{ij} , compuesta por 50 trabajos y 5 máquinas paralelas no relacionadas. Tras resolver el problema se ha obtenido un valor óptimo del *makespan* de:

$$C_{\text{máx}} = 168$$

Este valor constituye el óptimo global del modelo base y será el punto de referencia para evaluar cómo se comportan los tres métodos de resolución empleados a lo largo del trabajo.

4. Metodología

Tras resolver el modelo base mediante el método exacto con *OpenSolver*, el análisis se amplía en dos direcciones complementarias. Por un lado, se introducen de forma progresiva restricciones adicionales que incrementan la complejidad del problema. Por otro, se comparan tres enfoques de resolución distintos para cada nivel de complejidad.

4.1. Enfoques de resolución

A lo largo del trabajo se emplean tres métodos de resolución:

1. **Modelo exacto CP-SAT:** Solver de programación por restricciones desarrollado por Google OR-Tools. Se utiliza como método exacto de referencia, especialmente a partir de la incorporación de restricciones como fechas de disponibilidad, no solapamiento o recursos acumulativos, donde el modelo en Excel deja de ser viable debido al crecimiento del número de variables y restricciones. CP-SAT permite modelar este tipo de problemas de forma eficiente, aunque su rendimiento queda limitado por el tiempo de cómputo disponible en instancias de tamaño moderado. En este trabajo se ejecuta con un límite de 1000 segundos.
2. **Algoritmo Genético (AG):** Metaheurística basada en la evolución de una población de soluciones codificadas como permutaciones globales de trabajos. Detallado en la Sección 5.
3. **Iterated Greedy (IG):** Metaheurística basada en ciclos de destrucción y reconstrucción de soluciones combinados con búsqueda local por inserción. Detallada en la Sección 7.

4.2. Estructura de extensiones

Las extensiones se construyen de forma **acumulativa**: cada una incorpora las restricciones de la anterior y añade una nueva capa de complejidad. La progresión es la siguiente:

- **Extensión 0** — $R \parallel C_{\text{máx}}$: modelo base sin restricciones adicionales, utilizado para validar los tres métodos.
- **Extensión 1** — $R \mid r_j \mid C_{\text{máx}}$: incorpora fechas de disponibilidad r_j .
- **Extensión 2** — $R \mid r_j, s_{ijk} \mid C_{\text{máx}}$: añade tiempos de setup dependientes de la secuencia y la máquina.
- **Extensión 3** — $R \mid r_j, s_{ijk}, \text{prec} \mid C_{\text{máx}}$: incorpora relaciones de precedencia entre trabajos.
- **Extensión 4** — $R \mid r_j, s_{ijk}, \text{prec}, \text{res} \mid C_{\text{máx}}$: añade disponibilidad de recursos de personal.
- **Extensión 5** — $R \mid r_j, s_{ijk}, \text{prec}, \text{res}, d_j \mid \sum w_j T_j$: cambia el objetivo a tardanza ponderada.

4.3. Generación de datos

Los parámetros adicionales necesarios para cada extensión se generan de forma sintética a partir de distribuciones aleatorias controladas mediante semillas fijas, lo que garantiza la reproducibilidad completa de los resultados.

En primer lugar, las fechas de disponibilidad r_j se generan siguiendo un esquema estratificado con el objetivo de simular distintos niveles de urgencia en los trabajos. En concreto, el 40 % de los trabajos está disponible en instantes tempranos (entre 0 y 5), el 35 % en instantes intermedios (entre 6 y 15) y el 25 % restante en instantes tardíos (entre 16 y 30). Posteriormente, estos valores se mezclan aleatoriamente para evitar patrones artificiales en el orden de los trabajos.

Los tiempos de setup s_{ijk} , que representan el coste de pasar del trabajo j al trabajo k en la máquina i , se generan a partir de una distribución uniforme discreta entre 1 y el 20 % de la media global de los tiempos de proceso, denotada por $\bar{p}$. Además, se impone que $s_{ijj} = 0$, ya que no existe coste de preparación al procesar un trabajo tras sí mismo.

En cuanto a las fechas de entrega d_j , estas se calculan como:

$$d_j = r_j + \alpha_j \cdot \bar{p}_j$$

donde $\bar{p}_j$ es el tiempo medio de proceso del trabajo j considerando todas las máquinas, y α_j es un factor aleatorio extraído de una distribución uniforme en el intervalo (1,2,1,8). De este modo, se generan plazos alcanzables pero con distinta holgura, introduciendo variabilidad en la dificultad del problema.

Los pesos w_j , utilizados en la función de tardanza ponderada, se asignan de forma aleatoria mediante una distribución uniforme discreta en el conjunto $\{1, 2, 3, 4, 5\}$, representando distintos niveles de prioridad entre los trabajos.

Las relaciones de precedencia se generan como un conjunto de 25 pares ordenados (j, k) que indican que el trabajo j debe completarse antes de que el trabajo k pueda comenzar. Para evitar restricciones incompatibles entre sí que harían el problema irresoluble, únicamente se consideran pares con $j < k$. De este modo no puede darse el caso de que dos trabajos se esperen mutuamente, garantizando que siempre exista al menos una programación factible.

Por último, en la extensión con restricciones de recursos, cada trabajo j requiere un número de operarios h_j generado aleatoriamente entre 1 y 3. La capacidad total de recursos H se define como:

$$H = \left\lfloor 0,6 \cdot \frac{\sum_{j=1}^N h_j}{M} \right\rfloor$$

lo que introduce una restricción realista de disponibilidad de personal sin hacer el problema excesivamente restrictivo.

Por tanto, este procedimiento permite generar instancias progresivamente más complejas y cercanas a entornos productivos reales, manteniendo al mismo tiempo el control experimental necesario para comparar los distintos métodos de resolución.

Tabla 1: Semillas utilizadas en la generación de parámetros

Parámetro	Semilla
Release dates r_j	123
Setups s_{ijk}	42
Due dates d_j	43
Pesos w_j	44
Precedencias	45
Recursos	46

4.4. Métrica de comparación

Para evaluar la calidad de los resultados se utiliza el *Relative Percentage Deviation* (RPD), que mide la desviación porcentual de un método respecto a la mejor solución encontrada:

$$\text{RPD}(\%) = \frac{C_{\text{método}} - C_{\text{referencia}}}{C_{\text{referencia}}} \cdot 100$$

Un valor positivo indica que el método está por encima de la referencia. Los parámetros de los algoritmos se fijan antes de la ejecución y no se modifican entre extensiones, garantizando así una comparación homogénea.

5. Algoritmo Genético

Para resolver el problema se ha implementado un algoritmo genético inspirado en la literatura sobre máquinas paralelas no relacionadas con tiempos de preparación dependientes de la secuencia (Vallada & Ruiz, 2011). En este enfoque, cada solución se representa mediante una permutación global de trabajos. Esta secuencia establece el orden en el que los trabajos serán considerados durante la construcción de la programación.

Representación

Cada individuo del algoritmo se define como una permutación $\pi = (j_1, j_2, \dots, j_n)$. Esta permutación no representa directamente una programación, sino el orden en el que los trabajos son evaluados para construirla.

Función de evaluación

Cada secuencia se transforma en una programación factible mediante una heurística constructiva de tipo greedy. La permutación se recorre de izquierda a derecha y, para cada trabajo j , se evalúa su asignación a todas las máquinas disponibles.

En cada máquina i se calcula el instante de inicio y finalización:

$$\text{inicio}_{ij} = \text{disponible}_i \quad \text{fin}_{ij} = \text{inicio}_{ij} + p_{ij}$$

El trabajo se asigna a la máquina que produce el menor tiempo de finalización. En caso de empate, se selecciona la máquina de menor índice. De este modo, cada permutación se traduce en una solución factible cuyo valor viene dado por el makespan:

$$C_{\text{máx}} = \max_i \{C_i\}$$

Operadores

Los operadores empleados siguen la estructura habitual de los algoritmos genéticos basados en permutaciones para problemas de programación, combinando diversidad poblacional y mecanismos de intensificación (Vallada & Ruiz, 2011).

1. **Inicialización:** La población inicial se construye de forma mixta. Un 65 % de los individuos se obtiene a partir de una permutación base ordenada, que se perturba mediante 12 intercambios aleatorios de posiciones para generar soluciones diferentes pero cercanas a dicha secuencia inicial. El 35 % restante se genera de forma completamente aleatoria, con el fin de introducir mayor diversidad en la población.
2. **Selección:** La selección de padres se realiza mediante torneo de 3 individuos. En cada torneo se eligen aleatoriamente tres soluciones de la población y se selecciona como progenitor aquella que presenta mejor valor de la función objetivo. Este procedimiento favorece la supervivencia de las mejores soluciones, pero mantiene al mismo tiempo cierto grado de diversidad.
3. **Cruce:** El operador de cruce utilizado es el cruce ordenado OX. A partir de dos padres, se selecciona un segmento de uno de ellos que se conserva en el hijo, y el resto de posiciones se completa con los trabajos del otro padre respetando su orden relativo. De este modo, se generan dos hijos válidos en una única operación de cruce, manteniendo la estructura de permutación.
4. **Mutación:** Sobre cada hijo se aplican, de forma independiente, dos posibles operadores de mutación. El primero consiste en intercambiar la posición de dos trabajos dentro de la permutación (*swap*). El segundo consiste en seleccionar un bloque consecutivo de trabajos e invertir su orden dentro de la secuencia.
5. **Elitismo:** Los 4 mejores individuos de cada generación pasan directamente a la siguiente sin sufrir modificaciones, de tal forma que las mejores soluciones encontradas hasta ese momento no se pierdan durante el proceso evolutivo.

Parámetros

Los parámetros del algoritmo se recogen en la Tabla 2 y se han fijado tomando como referencia la configuración propuesta por Vallada y Ruiz, 2011, adaptándola al objetivo y al diseño experimental de este trabajo.

Tabla 2: Parámetros del Algoritmo Genético

Parámetro	Valor
Tamaño de población	160
Probabilidad de cruce	0,92
Probabilidad de mutación swap	0,30
Probabilidad de mutación inversión	0,22
Elitismo	4
Máximo de generaciones	3000
Paciencia (generaciones sin mejora)	350
Semilla	9

Adaptación a las extensiones

La estructura del algoritmo permanece invariante a lo largo de todas las extensiones. La única modificación se realiza en la función de evaluación, que incorpora progresivamente los nuevos parámetros (r_j , s_{ijk} , precedencias, recursos) al calcular el tiempo de inicio en cada máquina.

En presencia de precedencias, cada permutación se repara antes de su evaluación para garantizar la factibilidad. Cuando se incorporan *release dates*, la población inicial se ordena según (r_j, j) , lo que permite partir de soluciones de mayor calidad.

En la Extensión 5, la función de evaluación deja de devolver el *makespan* y pasa a calcular la tardanza ponderada $\sum w_j T_j$, siendo este el único cambio adicional respecto a la Extensión 4.

6. Modelo exacto: CP-SAT

CP-SAT es un solver de programación por restricciones desarrollado por Google OR-Tools, especialmente orientado a la resolución de problemas combinatorios complejos como los de *scheduling* (Perron & Didier, 2024; Perron & Furnon, 2024). A diferencia de los solvers de programación lineal entera clásicos, CP-SAT trabaja directamente con variables enteras y dispone de mecanismos de propagación y búsqueda, así como de restricciones globales especializadas para modelar no solapamientos, precedencias y recursos acumulativos (Gedik et al., 2018; Perron & Didier, 2024; Yunusoglu & Topaloglu Yildiz, 2022). La propia documentación oficial de OR-Tools destaca que CP-SAT opera exclusivamente sobre modelos enteros, por lo que, cuando existen coeficientes no enteros, estos deben escalarse previamente (Perron & Didier, 2024; Perron & Furnon, 2024).

Su utilización en este trabajo se justifica a partir de la Extensión 1, dado que la incorporación de *release dates* obliga a introducir variables de tiempo de inicio y restricciones de no solapamiento entre pares de trabajos en cada máquina, lo que incrementa rápidamente el tamaño del modelo. En este contexto, CP-SAT permite representar de forma natural este tipo de restricciones y obtener soluciones factibles de referencia (Gedik et al., 2018; Yunusoglu & Topaloglu Yildiz, 2022). No obstante, en problemas de tamaño moderado con múltiples restricciones acumuladas, la certificación de optimalidad no siempre resulta alcanzable dentro del tiempo de cómputo fijado (Perron & Didier, 2024).

Formulación general

El modelo se escala a enteros mediante un factor 100 para evitar errores de redondeo en los tiempos (Perron & Didier, 2024; Perron & Furnon, 2024). Para garantizar el no solapamiento entre trabajos en una misma máquina, se consideran pares de trabajos $j < k$ y se introducen dos variables binarias:

- Y_{ijk} , que indica el orden de ejecución entre los trabajos j y k en la máquina i .
- Z_{ijk} , que indica si ambos trabajos se asignan a la misma máquina.

A partir de estas variables, el no solapamiento se modela mediante restricciones de tipo Big-M:

$$\begin{aligned} S_k &\geq C_j + s_{ijk} - M^*(1 - Y_{ijk}) - M^*(1 - Z_{ijk}) \\ S_j &\geq C_k + s_{ikj} - M^*Y_{ijk} - M^*(1 - Z_{ijk}) \end{aligned}$$

De este modo, cuando ambos trabajos quedan asignados a la misma máquina, el modelo fuerza que uno de ellos empiece después de la finalización del otro, teniendo en cuenta además el correspondiente tiempo de preparación. Si no se consideran tiempos de setup, los términos s_{ijk} y s_{ikj} toman valor cero, y las restricciones garantizan únicamente la secuenciación temporal correcta, es decir, el no solapamiento.

Para modelar la disponibilidad limitada de personal se emplea la restricción global `AddCumulative` de OR-Tools, que permite imponer que, en cualquier instante, la suma de operarios requeridos por los trabajos en ejecución no supere la capacidad total disponible H (Perron & Furnon, 2024; Yunusoglu & Topaloglu Yildiz, 2022).

Parámetros

Tabla 3: Parámetros del modelo CP-SAT

Parámetro	Valor
Tiempo límite	1000 s
Hilos de búsqueda	1
Escala entera	100
Semilla	123

7. Iterated Greedy

El Iterated Greedy (IG) es una metaheurística introducida por Ruiz y Stützle, 2007, basada en ciclos de destrucción parcial de la solución actual seguidos de una reconstrucción greedy y una búsqueda local. Posteriormente, este enfoque ha sido adaptado con éxito al problema de máquinas paralelas no relacionadas, mostrando un comportamiento muy competitivo en este tipo de entornos (Fanjul-Peyro & Ruiz, 2010). A pesar de su planteamiento sencillo, el método combina bien exploración y mejora local, lo que explica su buen comportamiento en problemas de *scheduling* con elevada complejidad combinatoria.

Estructura del algoritmo

La versión implementada en este trabajo sigue la lógica general del esquema Iterated Greedy, combinando destrucción, reconstrucción y búsqueda local sobre soluciones representadas mediante permutaciones de trabajos (Fanjul-Peyro & Ruiz, 2010; Ruiz & Stützle, 2007).

1. **Solución inicial:** Se construye sobre una permutación aleatoria de los trabajos y se aplica búsqueda local por inserción hasta que no se encuentre mejora.
2. **Destrucción:** Se eliminan aleatoriamente $d = 5$ trabajos de la solución actual, equivalente al 10 % de los trabajos.
3. **Reconstrucción:** Los trabajos eliminados se reinsertan de forma greedy, uno a uno, en la posición que minimiza el objetivo sobre la solución parcial.
4. **Búsqueda local:** Sobre la solución reconstruida se aplica búsqueda local por inserción exhaustiva hasta convergencia.
5. **Aceptación:** Se acepta la nueva solución solo si mejora estrictamente a la mejor conocida.

6. **Criterio de parada:** El proceso se repite hasta 800 iteraciones o hasta que 200 iteraciones consecutivas no produzcan mejora.

Parámetros

Los parámetros utilizados se muestran en la Tabla 4.

Tabla 4: Parámetros del Iterated Greedy

Parámetro	Valor
Trabajos destruidos por iteración (d)	5
Máximo de iteraciones	800
Paciencia (iteraciones sin mejora)	200
Semilla	42

Adaptación a las extensiones

Al igual que en el caso del algoritmo genético, la lógica general del IG permanece invariante a lo largo de todas las extensiones. La principal modificación se realiza en la función de evaluación, que se actualiza progresivamente para incorporar las nuevas restricciones del problema. Cuando se añaden *release dates*, la solución inicial pasa a ordenarse según (r_j, j) para partir de una configuración de mayor calidad. En presencia de precedencias, tanto la reconstrucción parcial como la búsqueda local garantizan la factibilidad reparando la permutación cuando es necesario.

En la Extensión 5, la función de evaluación deja de devolver el *makespan* y pasa a calcular la tardanza ponderada $\sum w_j T_j$, siendo este el único cambio adicional respecto a la Extensión 4.

8. Extensión 0: Modelo base $R \parallel C_{\text{máx}}$

La Extensión 0 aplica los tres métodos al problema base sin restricciones adicionales. Su objetivo es validar los algoritmos implementados y establecer una referencia computacional inicial. En particular, si la heurística constructiva del algoritmo genético y del Iterated Greedy está correctamente diseñada, ambos métodos deberían ser capaces de aproximarse al óptimo conocido del problema, $C_{\text{máx}} = 168$, obtenido mediante OpenSolver (Mason, 2012).

8.1. Resultados computacionales

Tabla 5: Resultados para el problema $R \parallel C_{\text{máx}}$

Método	Estado	$C_{\text{máx}}$	Tiempo (s)
IG	Metaheurística	168,00	78,86
Genético	Metaheurística	170,00	9,26
CP-SAT	Factible no óptimo	200,00	1000,08

$$\begin{aligned} \text{RPD}_1(\%) &= \frac{170,00 - 168,00}{168,00} \cdot 100 = +1,19\% \quad (\text{Genético vs. IG}) \\ \text{RPD}_2(\%) &= \frac{200,00 - 168,00}{168,00} \cdot 100 = +19,05\% \quad (\text{CP-SAT vs. IG}) \\ \text{RPD}_3(\%) &= \frac{200,00 - 170,00}{170,00} \cdot 100 = +17,65\% \quad (\text{CP-SAT vs. Genético}) \end{aligned}$$

El resultado más destacable es que el Iterated Greedy alcanza exactamente el óptimo global, $C_{\text{máx}} = 168$, reproduciendo el mismo valor obtenido con OpenSolver. Este resultado valida la correcta implementación de la heurística constructiva empleada. El algoritmo genético obtiene un makespan de 170, situándose únicamente un 1,19 % por encima del óptimo, lo que confirma también su buen comportamiento en el modelo base.

Por su parte, el CP-SAT solo alcanza un valor de 200 dentro del límite temporal fijado. Aunque se trata de un método exacto, incluso en esta versión básica del problema su capacidad de exploración queda condicionada por el tiempo disponible. Por ello, a partir de la Extensión 1 se utilizará como método exacto de referencia en términos de solución factible obtenida dentro de un horizonte de cómputo limitado, y no necesariamente como garantía práctica de optimalidad.

Tabla 6: Secuencia de trabajos por máquina — Extensión 0

Máq.	IG (168)	Genético (170)	CP-SAT (200)
1	13, 27, 30, 22, 31, 33, 15	37, 31, 42, 22, 15, 30, 13, 47	1, 31, 30, 15, 33, 13, 22
2	12, 14, 18, 40, 46, 23, 43, 25, 6, 7	14, 40, 23, 25, 27, 7, 18, 32, 6, 12, 43	10, 23, 6, 7, 12, 27, 26, 14, 25, 43, 18
3	17, 36, 44, 48, 39, 29, 49, 32, 10, 8, 50	36, 39, 48, 29, 17, 49, 5, 10, 34, 8, 44	48, 50, 39, 5, 21, 36, 44, 32, 16, 17, 49
4	42, 11, 21, 37, 19, 35, 47, 26, 3, 2, 5	19, 35, 26, 50, 21, 11, 3, 2, 33	19, 3, 2, 8, 11, 37, 35, 47, 42
5	4, 34, 24, 9, 1, 45, 20, 28, 38, 16, 41	9, 4, 45, 28, 41, 38, 20, 16, 1, 24, 46	9, 29, 38, 4, 45, 34, 28, 20, 40, 41, 24, 46

9. Extensión 1: Release dates r_j

En esta primera ampliación se incorporan *release dates* r_j , que representan el instante más temprano en el que cada trabajo j está disponible para comenzar su procesamiento. Esta restricción modela situaciones reales en las que los trabajos no están todos disponibles en el instante inicial. El problema pasa a ser:

$$R \mid r_j \mid C_{\text{máx}}$$

9.1. Nuevos parámetros y restricciones

Se añade el parámetro r_j (instante de disponibilidad del trabajo j) y la variable $S_j \geq 0$ (instante de inicio). La nueva restricción impone:

$$S_j \geq r_j \quad \forall j \in J$$

9.2. Adaptación de los métodos

En el Genético y el IG, la función constructiva se actualiza incorporando r_j en el cálculo del inicio:

$$\text{inicio}_{ij} = \text{máx}(\text{disponible}_i, r_j)$$

Además, la población inicial pasa a ordenarse por (r_j, j) en lugar de aleatoriamente, lo que permite partir de soluciones de mayor calidad cuando los trabajos tienen distintas fechas de disponibilidad. En CP-SAT se añade directamente la restricción $S_j \geq r_j$ al modelo.

9.3. Resultados computacionales

Tabla 7: Resultados para el problema $R \mid r_j \mid C_{\text{máx}}$

Método	Estado	$C_{\text{máx}}$	Tiempo (s)
IG	Metaheurística	169,00	83,45
Genético	Metaheurística	171,00	6,74
CP-SAT	Factible no óptimo	194,00	1000,06

$$\text{RPD}_1(\%) = \frac{171,00 - 169,00}{169,00} \cdot 100 = +1,18\% \quad (\text{Genético vs. IG})$$

$$\text{RPD}_2(\%) = \frac{194,00 - 169,00}{169,00} \cdot 100 = +14,79\% \quad (\text{CP-SAT vs. IG})$$

$$\text{RPD}_3(\%) = \frac{194,00 - 171,00}{171,00} \cdot 100 = +13,45\% \quad (\text{CP-SAT vs. Genético})$$

La incorporación de *release dates* incrementa ligeramente el *makespan* respecto al modelo base. El IG obtiene 169, apenas una unidad de tiempo por encima del óptimo del modelo sin *release dates*. El algoritmo genético alcanza 171, manteniéndose próximo al valor obtenido por IG. El CP-SAT devuelve 194 tras 1000 segundos sin certificar optimalidad, lo que confirma que el método exacto necesita tiempos de cómputo muy superiores para instancias de este tamaño.

Tabla 8: Secuencia de trabajos por máquina — Extensión 1

Máq.	IG (169)	Genético (171)	CP-SAT (194)
1	33, 22, 15, 30, 42, 31	22, 33, 15, 30, 42, 31	22, 31, 40, 33, 15, 30
2	25, 7, 12, 27, 21, 43, 40, 6, 18, 23	25, 6, 43, 7, 12, 40, 23, 27, 14, 49	19, 18, 27, 43, 14, 6, 7, 12, 23, 25
3	13, 32, 14, 8, 50, 39, 44, 49, 17, 36, 48, 10	8, 32, 48, 36, 50, 39, 29, 34, 44, 13, 10, 17	13, 8, 44, 32, 49, 10, 50, 5, 39, 17, 37, 36
4	19, 2, 35, 3, 5, 47, 45, 11, 37	21, 19, 26, 37, 35, 18, 2, 11, 5, 47, 3	26, 3, 47, 9, 11, 2, 21, 35, 42
5	1, 41, 20, 16, 4, 38, 46, 28, 24, 29, 34, 9, 26	1, 24, 4, 46, 38, 20, 45, 16, 9, 28, 41	1, 38, 20, 34, 24, 48, 46, 28, 4, 41, 29, 16, 45

10. Extensión 2: Tiempos de setup dependientes de la secuencia

En esta segunda ampliación se introducen tiempos de preparación dependientes de la secuencia (*setup times*). El tiempo s_{ijk} representa el tiempo necesario en la máquina i para pasar del trabajo j al trabajo k . Este fenómeno es habitual en entornos donde es necesario cambiar herramientas, limpiar equipos o ajustar parámetros entre trabajos distintos, y constituye una de las extensiones más relevantes en la literatura de *scheduling* por su impacto directo sobre la dificultad del problema (Allahverdi, 2015; Allahverdi et al., 2008). El problema pasa a ser:

$$R \mid r_j, s_{ijk} \mid C_{\text{máx}}$$

10.1. Nuevos parámetros y restricciones

Se añade el parámetro s_{ijk} y las restricciones de secuenciación: si el trabajo j precede al trabajo k en la máquina i , debe cumplirse:

$$S_k \geq C_j + s_{ijk}$$

10.2. Adaptación de los métodos

En el algoritmo genético y en el Iterated Greedy, la función constructiva actualiza el cálculo del inicio incorporando tanto la fecha de disponibilidad como el tiempo de preparación asociado al último trabajo procesado en cada máquina:

$$\text{inicio}_{ij} = \text{máx}(\text{disponible}_i + s_{i, \text{ult}_i, j}, r_j)$$

donde ult_i representa el último trabajo procesado en la máquina i . En CP-SAT se incorporan las restricciones de no solapamiento con setup mediante las variables binarias Y_{ijk} y Z_{ijk} descritas en la Sección 6.

10.3. Resultados computacionales

Tabla 9: Resultados para el problema $R \mid r_j, s_{ijk} \mid C_{\text{máx}}$

Método	Estado	$C_{\text{máx}}$	Tiempo (s)
IG	Metaheurística	195,00	94,65
Genético	Metaheurística	209,00	5,80
CP-SAT	Factible no óptimo	670,00	1000,03

$$\text{RPD}_1(\%) = \frac{209,00 - 195,00}{195,00} \cdot 100 = +7,18\% \quad (\text{Genético vs. IG})$$

$$\text{RPD}_2(\%) = \frac{670,00 - 195,00}{195,00} \cdot 100 = +243,59\% \quad (\text{CP-SAT vs. IG})$$

$$\text{RPD}_3(\%) = \frac{670,00 - 209,00}{209,00} \cdot 100 = +220,57\% \quad (\text{CP-SAT vs. Genético})$$

La incorporación de tiempos de setup incrementa notablemente la dificultad combinatoria del problema, tal y como señalan Allahverdi et al., 2008 y Allahverdi, 2015. En nuestros resultados, CP-SAT devuelve una solución de muy baja calidad (670) en el tiempo disponible, lo que evidencia la dificultad añadida que introducen las variables de secuenciación Y_{ijk} y Z_{ijk} para una instancia de 50 trabajos y 5 máquinas. Las metaheurísticas, en cambio, mantienen un rendimiento mucho más sólido: el IG obtiene 195 y el Genético 209, ambos en tiempos de cómputo reducidos. La brecha entre ambas metaheurísticas crece ligeramente respecto a extensiones anteriores, lo que sugiere que la búsqueda local por inserción del IG resulta especialmente eficaz para amortiguar el efecto acumulado de los setups.

Tabla 10: Secuencia de trabajos por máquina — Extensión 2

Máq.	IG (195)	Genético (209)	CP-SAT (670)
1	22, 13, 30, 15, 31, 42, 33	33, 22, 15, 31, 30, 40	6, 49, 47, 45, 44, 31, 22, 12, 7, 3
2	6, 14, 27, 12, 43, 7, 18, 25, 23, 5, 40	6, 27, 7, 26, 5, 49, 23, 14, 12, 25, 43	43, 48, 23, 20, 18, 17, 15, 13, 1
3	49, 8, 39, 36, 44, 48, 29, 10, 17, 50, 32	13, 50, 32, 48, 44, 17, 29, 34, 10, 8, 36, 39	14, 16, 37, 35, 33, 10, 50, 42, 39, 36, 32, 24
4	19, 11, 35, 45, 47, 3, 2, 26, 37, 21	19, 35, 21, 3, 47, 37, 18, 2, 11, 42	2, 9, 4, 38, 30, 29, 28, 21, 11
5	41, 1, 20, 24, 38, 46, 16, 28, 4, 34, 9	1, 45, 20, 4, 28, 41, 46, 9, 24, 38, 16	19, 5, 41, 26, 8, 46, 40, 34, 27, 25

11. Extensión 3: Relaciones de precedencia

La Extensión 3 incorpora relaciones de precedencia entre trabajos, que imponen que determinados trabajos no pueden comenzar hasta que otros hayan finalizado. Este tipo de restricción modela dependencias tecnológicas o de proceso frecuentes en entornos de fabricación y constituye una característica habitual en problemas de programación con restricciones estructurales (Afzalirad & Rezaeian, 2016; Anil Kumar et al., 2009). El problema pasa a ser:

$$R \mid r_j, s_{ijk}, \text{prec} \mid C_{\text{máx}}$$

11.1. Nuevos parámetros y restricciones

Se define el conjunto de pares de precedencia $\mathcal{P} = \{(a, b) : a \text{ debe finalizar antes de que } b \text{ comience}\}$, con 25 pares generados de manera que no haya ciclos que provoquen la infactibilidad del problema. La restricción añadida para ello es:

$$S_b \geq C_a \quad \forall (a, b) \in \mathcal{P}$$

11.2. Adaptación de los métodos

En el algoritmo genético y en el Iterated Greedy, la función constructiva actualiza el tiempo mínimo de inicio de cada trabajo:

$$t_{\text{mín}}(j) = \max\left(r_j, \max_{a:(a,j) \in \mathcal{P}} C_a\right)$$

Además, cada permutación se repara antes de la evaluación para garantizar que los trabajos predecesores aparezcan antes que sus sucesores. En CP-SAT se añaden directamente las restricciones $S_b \geq C_a$.

11.3. Resultados computacionales

Tabla 11: Resultados para el problema $R \mid r_j, s_{ijk}, \text{prec} \mid C_{\max}$

Método	Estado	$C_{\max}$	Tiempo (s)
IG	Metaheurística	197,00	214,80
Genético	Metaheurística	219,00	11,70
CP-SAT	Factible no óptimo	230,00	1000,04

$$\text{RPD}_1(\%) = \frac{219,00 - 197,00}{197,00} \cdot 100 = +11,17\% \quad (\text{Genético vs. IG})$$

$$\text{RPD}_2(\%) = \frac{230,00 - 197,00}{197,00} \cdot 100 = +16,75\% \quad (\text{CP-SAT vs. IG})$$

$$\text{RPD}_3(\%) = \frac{230,00 - 219,00}{219,00} \cdot 100 = +5,02\% \quad (\text{CP-SAT vs. Genético})$$

La introducción de precedencias tiene un impacto moderado sobre el makespan respecto a la Extensión 2. El Iterated Greedy obtiene 197, mientras que CP-SAT mejora relativamente su posición hasta 230. Este comportamiento puede explicarse porque las relaciones de precedencia introducen estructura adicional en el problema, reduciendo parcialmente el espacio de búsqueda y facilitando que el solver explore soluciones más ordenadas. La brecha entre el Genético (219) y el IG (197) se amplía, lo que sugiere que la búsqueda local por inserción del IG resulta más eficaz para explotar esta estructura de dependencias entre trabajos.

Tabla 12: Secuencia de trabajos por máquina — Extensión 3

Máq.	IG (197)	Genético (219)	CP-SAT (230)
1	30, 22, 47, 24, 13, 15, 31	30, 24, 33, 13, 15, 31	33, 35, 30, 13, 22, 15, 31, 44
2	43, 12, 14, 23, 6, 27, 18, 40, 25, 7, 26	7, 18, 32, 14, 6, 27, 5, 23, 40, 43, 12	7, 46, 32, 14, 21, 12, 23, 18, 42, 6
3	49, 32, 36, 8, 10, 5, 29, 50, 17, 39, 44	49, 17, 50, 36, 10, 8, 34, 48, 39, 44	49, 17, 8, 10, 48, 36, 43, 39, 50
4	19, 3, 2, 42, 33, 21, 11, 37, 35	19, 42, 3, 2, 11, 37, 47, 26, 21, 35, 22	3, 2, 19, 5, 47, 24, 11, 27, 26, 37
5	41, 28, 4, 48, 20, 1, 9, 38, 16, 46, 45, 34	29, 41, 4, 20, 46, 38, 1, 9, 16, 28, 25, 45	1, 28, 4, 9, 29, 45, 34, 20, 16, 40, 25, 41, 38

12. Extensión 4: Disponibilidad de recursos de personal

La Extensión 4 incorpora una restricción de recursos acumulativos asociada a la disponibilidad de personal. Cada trabajo j requiere h_j operarios durante su procesamiento, y el número total de operarios simultáneamente activos no puede superar la capacidad H . Este tipo de restricción resulta especialmente relevante en problemas de programación donde, además de las máquinas, intervienen recursos compartidos limitados (Afzalirad & Rezaeian, 2016; Yunusoglu & Topaloglu Yildiz, 2022). El problema pasa a ser:

$$R \mid r_j, s_{ijk}, \text{prec}, \text{res} \mid C_{\text{máx}}$$

12.1. Nuevos parámetros y restricciones

Se añaden los parámetros h_j (operarios requeridos por el trabajo j) y H (capacidad total de personal). La restricción acumulativa impone que en todo instante t :

$$\sum_{j \in \text{activos}(t)} h_j \leq H$$

12.2. Adaptación de los métodos

En el algoritmo genético y en el Iterated Greedy, antes de asignar un trabajo a una máquina se comprueba si existe suficiente personal disponible en el instante propuesto. Cuando no lo hay, el inicio del trabajo se retrasa hasta el primer instante en que se libera capacidad suficiente. En CP-SAT esta restricción se modela mediante la función global `AddCumulative` de OR-Tools.

12.3. Resultados computacionales

Tabla 13: Resultados para el problema $R \mid r_j, s_{ijk}, \text{prec}, \text{res} \mid C_{\text{máx}}$

Método	Estado	$C_{\text{máx}}$	Tiempo (s)
IG	Metaheurística	199,00	2191,15
Genético	Metaheurística	228,00	92,23
CP-SAT	Factible no óptimo	263,00	1000,07

$$\text{RPD}_1(\%) = \frac{228,00 - 199,00}{199,00} \cdot 100 = +14,57\% \quad (\text{Genético vs. IG})$$

$$\text{RPD}_2(\%) = \frac{263,00 - 199,00}{199,00} \cdot 100 = +32,16\% \quad (\text{CP-SAT vs. IG})$$

$$\text{RPD}_3(\%) = \frac{263,00 - 228,00}{228,00} \cdot 100 = +15,35\% \quad (\text{CP-SAT vs. Genético})$$

La incorporación de la restricción de recursos incrementa de forma muy notable el tiempo de ejecución del Iterated Greedy, que supera los 2191 segundos. Este aumento refleja el coste computacional asociado a verificar la disponibilidad de personal dentro de la búsqueda local. A pesar de ello, el IG obtiene la mejor solución, con un makespan de 199, superando con claridad al algoritmo genético (228) y a CP-SAT (263).

El elevado tiempo de ejecución del IG se explica porque la búsqueda local por inserción debe evaluar numerosas posiciones candidatas y, en cada una de ellas, comprobar además la factibilidad respecto al recurso compartido. Aun así, su mejor rendimiento en calidad de solución sugiere que esta exploración intensiva sigue siendo eficaz incluso cuando el problema incorpora restricciones adicionales sobre recursos.

Tabla 14: Secuencia de trabajos por máquina — Extensión 4

Máq.	IG (199)	Genético (228)	CP-SAT (263)
1	30, 22, 33, 42, 15, 31	33, 30, 15, 40, 22, 31	30, 47, 35, 45, 22, 27
2	6, 7, 23, 27, 49, 12, 43, 40, 25	43, 29, 23, 49, 18, 6, 25, 12, 27, 7	43, 12, 31, 14, 6, 15, 23, 24, 25
3	32, 48, 36, 14, 8, 10, 13, 50, 34, 44, 17, 39	8, 32, 48, 50, 17, 36, 14, 10, 13, 16, 39, 44	32, 5, 7, 33, 10, 42, 13, 17, 36, 44, 50, 39, 34
4	3, 2, 19, 5, 21, 18, 26, 11, 35, 37, 47	19, 42, 5, 3, 21, 2, 26, 47, 37, 11, 35	19, 4, 11, 2, 37, 18, 48, 3, 21
5	24, 9, 4, 20, 46, 45, 38, 1, 16, 41, 28, 29	41, 9, 34, 4, 46, 24, 20, 1, 28, 45, 38	49, 8, 9, 26, 41, 46, 1, 16, 28, 29, 20, 40, 38

13. Extensión 5: Tardanza ponderada $\sum w_j T_j$

La Extensión 5 modifica el objetivo del problema: en lugar de minimizar el makespan, se pasa a minimizar la tardanza ponderada total, definida como la suma de los retrasos respecto a las fechas de entrega d_j , ponderados por los pesos w_j . Este cambio hace que el problema sea más exigente, ya que ya no basta con terminar pronto, sino que también importa qué trabajos se retrasan y cuánto pesa cada uno de esos retrasos. El problema pasa a ser:

$$R \mid r_j, s_{ijk}, \text{prec}, \text{res}, d_j \mid \sum w_j T_j$$

13.1. Nuevos parámetros y restricciones

Se incorporan en esta extensión los parámetros d_j (fecha de entrega del trabajo j) y w_j (peso o prioridad del trabajo j), así como la variable $T_j \geq 0$ (tardanza del trabajo j). Las restricciones asociadas son:

$$T_j \geq C_j - d_j \quad \forall j \in J$$

$$T_j \geq 0 \quad \forall j \in J$$

Combinadas, equivalen a: $T_j = \max(0, C_j - d_j)$.

La función objetivo pasa a ser:

$$\min \sum_{j=1}^N w_j T_j$$

13.2. Adaptación de los métodos

La principal diferencia respecto a las extensiones anteriores es que la evaluación de las soluciones deja de basarse en el makespan y pasa a centrarse en la tardanza ponderada total.

En el algoritmo genético y en el Iterated Greedy, una vez construida la programación, se calcula el valor:

$$\sum_j w_j \max(0, C_j - d_j)$$

En CP-SAT se introducen variables $T_j \geq 0$ junto con las restricciones $T_j \geq C_j - d_j$, y la función objetivo se define directamente como la minimización de $\sum_j w_j T_j$.

13.3. Resultados computacionales

Tabla 15: Resultados para el problema $R \mid r_j, s_{ijk}, \text{prec}, \text{res}, d_j \mid \sum w_j T_j$

Método	Estado	$\sum w_j T_j$	Tiempo (s)
IG	Metaheurística	3060,00	5640,78
Genético	Metaheurística	4426,00	284,13
CP-SAT	Factible no óptimo	7361,00	1000,03

$$\text{RPD}_1(\%) = \frac{4426,00 - 3060,00}{3060,00} \cdot 100 = +44,64\% \quad (\text{Genético vs. IG})$$

$$\text{RPD}_2(\%) = \frac{7361,00 - 3060,00}{3060,00} \cdot 100 = +140,56\% \quad (\text{CP-SAT vs. IG})$$

$$\text{RPD}_3(\%) = \frac{7361,00 - 4426,00}{4426,00} \cdot 100 = +66,31\% \quad (\text{CP-SAT vs. Genético})$$

La minimización de la tardanza ponderada es, con diferencia, el escenario más exigente de todos los estudiados. Las diferencias entre métodos se amplían de forma muy notable respecto a las extensiones anteriores. CP-SAT obtiene un valor de 7361, más del doble que el alcanzado por el Iterated Greedy (3060), lo que pone de manifiesto la gran dificultad de resolver este problema de forma exacta cuando se combinan múltiples restricciones y un objetivo más exigente que el makespan.

El Iterated Greedy vuelve a proporcionar la mejor solución de los tres métodos, aunque a costa de un tiempo de ejecución muy elevado, superior a 5640 segundos. Este resultado refuerza la idea de que, en problemas altamente restringidos, la intensificación proporcionada por la búsqueda local sigue siendo muy eficaz para mejorar la calidad de las soluciones, aunque ello implique un coste computacional considerable.

Tabla 16: Secuencia de trabajos por máquina — Extensión 5

Máq.	IG (3060)	Genético (4426)	CP-SAT (7361)
1	33, 30, 22, 44, 31, 15, 37	22, 42, 31, 30, 15, 44, 40	42, 22, 15, 31, 24, 40, 44
2	43, 12, 6, 14, 32, 27, 18, 7, 25, 23, 40	27, 12, 6, 14, 25, 18, 43, 7, 32, 23	33, 12, 14, 18, 6, 27, 26, 47, 23, 21, 7
3	8, 29, 10, 13, 17, 36, 39, 50, 34, 48, 49	17, 29, 33, 8, 10, 13, 50, 39, 36, 48, 49	8, 29, 17, 5, 50, 10, 13, 34, 32, 39, 36, 37
4	3, 2, 19, 5, 47, 26, 42, 21, 11, 35	19, 5, 47, 2, 3, 21, 26, 11, 35, 37	35, 2, 3, 19, 46, 41, 25, 11
5	1, 28, 4, 20, 41, 16, 46, 9, 24, 38, 45	28, 41, 1, 9, 20, 4, 34, 16, 46, 24, 38, 45	1, 28, 9, 4, 20, 45, 16, 30, 48, 38, 43, 49

14. Conclusiones

Este trabajo ha abordado de forma progresiva un problema de programación de la producción en máquinas paralelas no relacionadas, partiendo del modelo base $R \parallel C_{\text{máx}}$ y ampliándolo hasta el caso más complejo $R \mid r_j, s_{ijk}, \text{prec}, \text{res}, d_j \mid \sum w_j T_j$. Para cada nivel de complejidad se han comparado tres enfoques de resolución: un modelo exacto basado en CP-SAT, un algoritmo genético y un algoritmo Iterated Greedy.

Los resultados obtenidos muestran que CP-SAT solo resulta competitivo en las extensiones de menor complejidad. En el modelo base y en la Extensión 1 proporciona soluciones factibles de referencia, aunque sin certificar optimalidad dentro del límite temporal fijado. Sin embargo, a partir de la incorporación de tiempos de preparación dependientes de la secuencia, precedencias y recursos compartidos, la calidad de las soluciones encontradas se deteriora notablemente. Esto pone de manifiesto la dificultad de aplicar métodos exactos a instancias de tamaño moderado cuando se acumulan múltiples restricciones operativas.

Por el contrario, las dos metaheurísticas mantienen un comportamiento mucho más robusto a lo largo de todas las extensiones. Tanto el algoritmo genético como el Iterated Greedy conservan la misma estructura general y adaptan las nuevas restricciones a través de la función de evaluación, lo que les proporciona una gran flexibilidad. Esto supone una ventaja clara, porque permite incorporar nuevas restricciones sin tener que rediseñar por completo el método.

Entre ambos enfoques metaheurísticos, el Iterated Greedy destaca como el método con mejor rendimiento global. Obtiene la mejor solución en todas las extensiones y, además, reproduce el óptimo global del modelo base, con $C_{\text{máx}} = 168$. Su combinación de destrucción-reconstrucción y búsqueda local por inserción permite equilibrar de forma eficaz diversificación e intensificación, lo que se traduce en soluciones de alta calidad incluso en los escenarios más exigentes.

Finalmente, los resultados confirman la importancia de las metaheurísticas para abordar problemas reales de programación de la producción. Cuando se combinan fechas de disponibilidad, tiempos de preparación, precedencias, recursos compartidos y objetivos basados en tardanza, los métodos exactos dejan de ser prácticos en tiempos de cómputo razonables. En este contexto, enfoques como el Iterated Greedy y, en menor medida, el algoritmo genético, constituyen herramientas especialmente valiosas para apoyar la toma de decisiones en sistemas productivos complejos.

Referencias

- Afzalirad, M., & Rezaeian, J. (2016). Resource-constrained unrelated parallel machine scheduling problem with sequence dependent setup times, precedence constraints and machine eligibility restrictions. *Computers & Industrial Engineering*, 98, 40-52.
- Allahverdi, A. (2015). The third comprehensive survey on scheduling problems with setup times/costs. *European journal of operational research*, 246(2), 345-378.
- Allahverdi, A., Ng, C. T., Cheng, T. E., & Kovalyov, M. Y. (2008). A survey of scheduling problems with setup times or costs. *European journal of operational research*, 187(3), 985-1032.
- Anil Kumar, V., Marathe, M. V., Parthasarathy, S., & Srinivasan, A. (2009). Scheduling on unrelated machines under tree-like precedence constraints. *Algorithmica*, 55(1), 205-226.
- Fanjul-Peyro, L., & Ruiz, R. (2010). Iterated greedy local search methods for unrelated parallel machine scheduling. *European Journal of Operational Research*, 207(1), 55-69.
- Gedik, R., Kalathia, D., Egilmez, G., & Kirac, E. (2018). A constraint programming approach for solving unrelated parallel machine scheduling problem. *Computers & Industrial Engineering*, 121, 139-149.
- Mason, A. J. (2012). OpenSolver for Excel - The Open Source Optimization Solver for Excel. *University of Auckland*. <https://opensolver.org/>
- Perron, L., & Didier, F. (2024, 7 de mayo). *CP-SAT* (Ver. v9.10). https://developers.google.com/optimization/cp/cp_solver/
- Perron, L., & Furnon, V. (2024, 7 de mayo). *OR-Tools* (Ver. v9.10). <https://developers.google.com/optimization/>
- Ruiz, R., & Stützle, T. (2007). A simple and effective iterated greedy algorithm for the permutation flowshop scheduling problem. *European journal of operational research*, 177(3), 2033-2049.
- Vallada, E., & Ruiz, R. (2011). A genetic algorithm for the unrelated parallel machine scheduling problem with sequence dependent setup times. *European Journal of Operational Research*, 211(3), 612-622.
- Yunusoglu, P., & Topaloglu Yildiz, S. (2022). Constraint programming approach for multi-resource-constrained unrelated parallel machine scheduling problem with sequence-dependent setup times. *International Journal of Production Research*, 60(7), 2212-2229.